Start at 9:35

Function

- print() print("Hello"), int(), float()
  input(), sum(), max(), min()
  range() → range(1, 2, 3)
- Our goal: to create our own functions

---

1) Create a function

get the values f(x, y)

Syntax: def function_name(optional parameters):
    statement
    statement
    statement

function-block

space/indentation → to indicate inside/part of the function

when function is called, the code inside will be executed!

2) Use functions

depend on params

↳ function_name(optional values)

call the function

Simple functions with no parameters

↳ Create a function to print my name ("Nut")

- Create: ✓
```
def print_myname():
    print("Nut")
```
→ Nut

- Usage: ✓
```
print_myname()
```

EX2: create a function that prints greeting and my names

✓
```
def greeting():
    print("Welcome!")
    print("Nut")
    print('Enjoy your stay.')
```

Created

Welcome
Nut
Enjoy your stay

```
print("Enjoy your stay.")        Enjoy your stay
greeting()
```
Usage

EX3 Create a function with a loop that print "X" 3 times

```
def loop_3x():
    for i in range(3):
        print("X")
loop_3x()
```
function-block

X
X
X

EX4 Multiple functions

```
def print_a():
    print("AAA")
def print_b():
    print("BBB")
def main():
    print_a()
    print_b()
main()
```
Create
Create
Create
call → main()

AAA

BBB

Function with one parameter

- Parameter is a variable that will store a value when the function is called with values. (e.g. function(value))
- These variables/parameters are only available inside the function.

EX A function that takes in a name and print the greeting with that name.

parameter / variable

```
def greeting(name):
    print("Welcome", name)
greeting("Nut")
```
① "Nut"
② "Nut"
Welcome Nut

* Note: if the function has a parameter, you must pass a value when you call the function.

---

* multiple parameters

function.

* multiple parameters
  - separate each parameter by a comma

```
def function_name(param1, param2, param3):
    statement
    statement

function_name(value1, value2, value3)
```

Ex)

```
def greeting(my_name, your_name):
    print("Hello", my_name)
    print("Hi", your_name)

greeting("Alice", "Bob")
```

"Alice" → my_name, "Bob" → your_name

Hello Alice
Hi Bob

Create

Call

* positional arguments

Key word Arguments

* Call the function with a value specifically for a parameter explicitly by using function(parameter_name = value)

Eg:

```
greeting(my_name="Alice", your_name="Bob")
```

* positions don't matter with keyword arguments

```
greeting(your_name="Bob", my_name="Alice")
```

* If you use both positional and keyword arguments, positional must come first!

```
greeting("Alice", your_name="Bob")      ✓
greeting(your_name="Bob", "Alice")      ✗
```

Default Values

* You can provide default values for parameters

Syntax:

```
def function_name(param="default"):
    statement
```

"Alice" default value

Syntax: def function_name(parameter = default):
statement

EX1

```
def greeting(my_name = "Nut"):
    print("Hello", my_name)

greeting()
greeting("Alice")
```

= "Alice" — default value

→ Hello Nut

No value provided
Use default.

→ Hello Alice

If you provide a value, the function will use it.

✱ default values must come after parameters with no default

EX2

```
def do_print(a, b="B", c="C"):
    print(a, b, c)

do_print("X")
do_print("X", "Y")
do_print("O", "U", "T")
```

do_print("X") → a="X", b="B", c="C" → X B C

do_print("X", "Y") → a="X", b="Y", c="C" → X Y C

do_print("O", "U", "T") → O U T

a='O', b='U', c='T'

EX3

```
def mean(x, y, z):
    avg = (x+y+z)/3
    print(avg)

mean(3, 4, 5)
```

x=3, y=4, z=5

(3+4+5)/3 → 4

→ 4

✱ Variables inside a function are only available inside of that function.
You cannot use variables inside a function from elsewhere. = local variables

You cannot call avg outside out of mean function.

```
def mean(x, y, z):
    avg = (x+y+z)/3
    print(avg)

def main():
```

✗ cannot access!!

```
def main():
    mean(1,1,1)
    print(avg)
main()
```

cannot access!!

error!

```
def mean(x,y,z):
    avg = (x+y+z)/3
    print(avg)
def main():
    avg = 5
    mean(1,1,1)
    print(avg)
main()
```

different variables!

avg is only in main

1

1

5

---

return statement

↳ return values out of the function and it will get out of function (= break)

EX1

```
def return_5():
    return 5
X = return_5()
print(X)
```

5

5

EX2

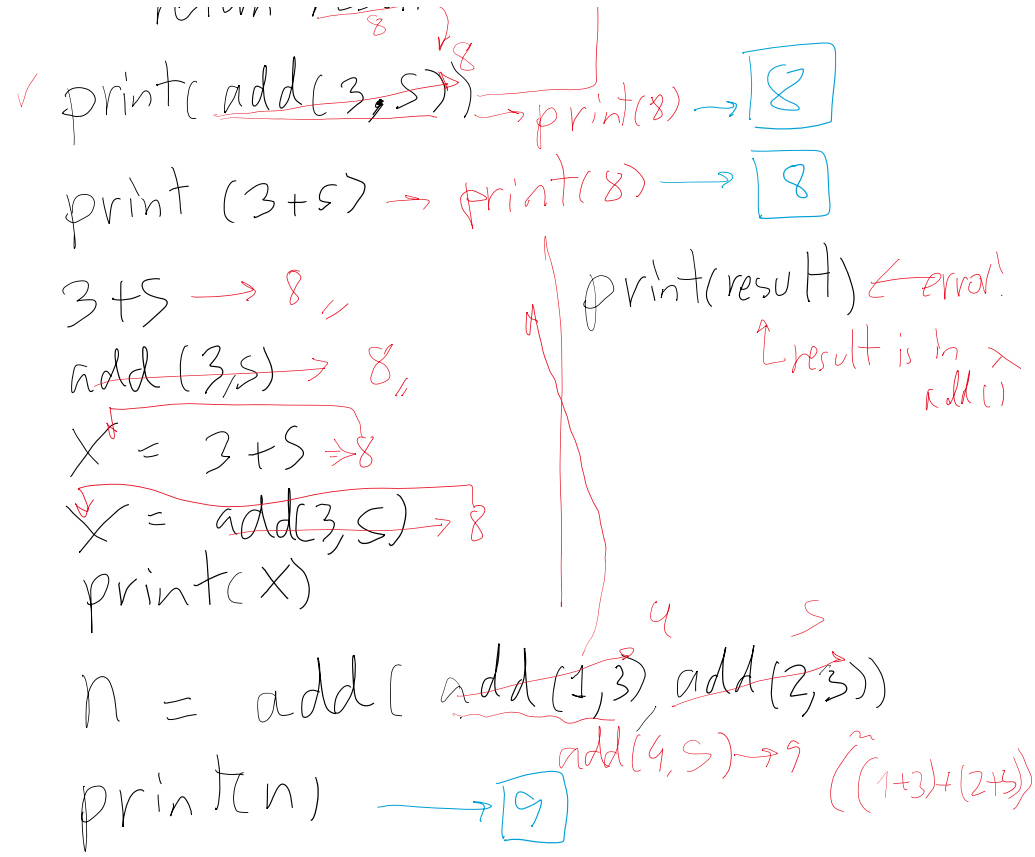
```
def add(a,b):
    result = a+b
    return result
print(add(3,5))
```

3+5 = 8

8

8

12



EX - linear function

$y = mx + c$

- function will take in three values:

  m, x, c

- function will return y.

```
def linear(m, x, c):
    y = m*x + c      → 1*2+3 = 5
    return y
print(linear(1,2,3)) → 5
```

EX let's create our sum function

. . . . . will take in a list of numbers

Ex let's create our own ...

↳ function will take in a list of numbers

↳ return the sum of all numbers

```
def our_sum(numbers):
    total = 0
    for num in numbers:
        totale = num
    return total
```

(numbers ← a list of numbers (e.g. [1,2,3]))

Multiple return

↳ you can return multiple values by separating them with commas.

syntax: return value1, value2, value3

Ex)

```
def mul_return():
    return 1, 2, 3
```

* It returns a tuple of values → (1, 2, 3)

```
def mul_return():
    return 1
    return 2
    return 3
```

not the same as (✗)

- return will get out of the function
- Therefore, it only returns 1!!

```
x = mul_return()        # (1,2,3)
print(x)        →  (1, 2, 3)
print(x[0])     →  1
```

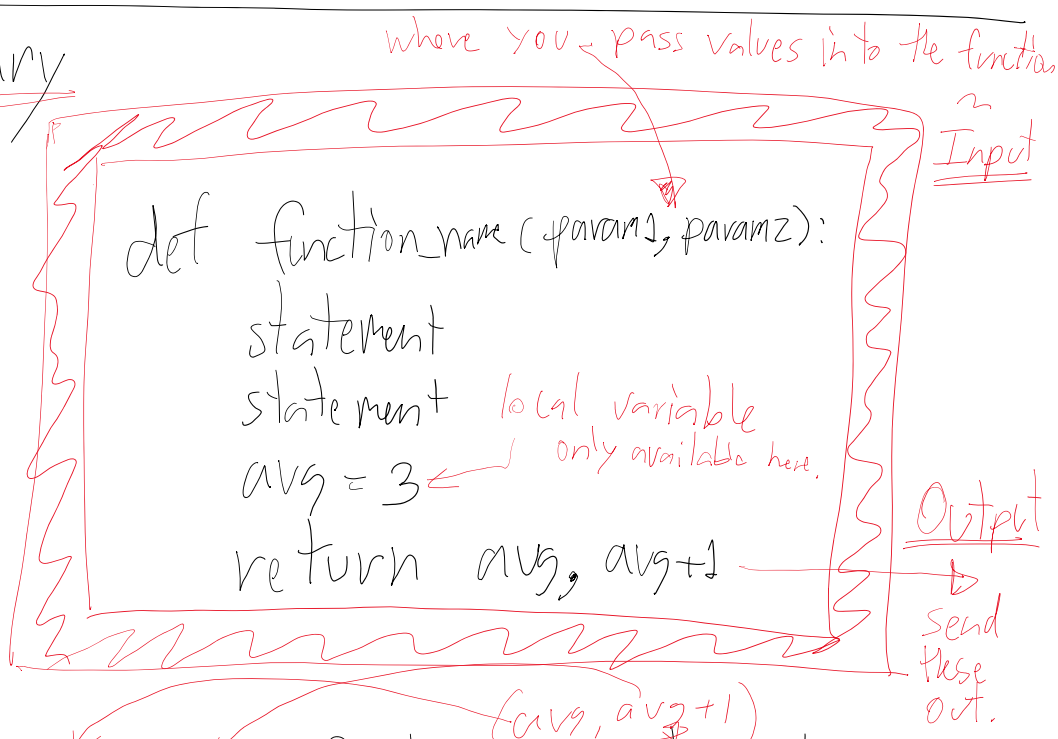
* Trick! you can have multiple variables for receive multiple values in a list/tuple

* Trick: you can have multiple ... for receive multiple values in a list/tuple

```
a, b, c = (1, 2, 3)   # [1, 2, 3]
print(b) ---> 2
print(a) ---> 1
a, b, c = multi_return()   (1, 2, 3)
```

* must have the same numbers of variables and values

Summary



Usage:

```
first, second = function_name(value1, value2)
                                (avg, avg+1)
results = function_name(value1, value2)
                        (avg, avg+1)
```